

Lecture Note: OOPs Concepts - Advanced



Nishut Suman

24 Dec, 2025 At 6:00 PM

Notes

Foundation

Recommended

Resource



Associated Lectures (1)



Description



Discussions



Lecture Note: OOPs Concepts - Advanced

Before You Begin

Before diving into advanced OOPs, recall the essentials from the previous session:

- You already know how to define a **class** and create **objects**.
- You understand the role of **self** in referring to instance attributes.
- You've worked with the constructor method `__init__()` for initialization.

This session extends those foundations by exploring how objects relate to each other, share behavior, and protect their internal data.

Prerequisites:

- Understanding of **class**, **object**, `__init__`, and **self**
- Basic Python syntax, variables, and functions

What you'll be able to do:

- Explain and apply **inheritance**, **method overriding**, and **method overloading**
- Use **super()** to access parent class methods safely
- Apply **encapsulation** and **abstraction** to design secure, modular systems
- Integrate all OOP concepts in one cohesive program

1. Introduction: What Are Advanced OOP Concepts and Why Should You Care?

Core Definition

Advanced OOP concepts - inheritance, method overriding, method overloading, encapsulation, and abstraction - help you design programs that are **modular, reusable, and secure**.

They represent how real-world entities interact and evolve in software.

A Simple Analogy

Think of your smartphone 📱 :

You can install new apps (**inheritance**), customize existing ones (**overriding**), and use the same button for multiple tasks (**overloading**).

But you don't see what's happening inside the operating system (**encapsulation and abstraction**).

This analogy helps visualize relationships and data hiding, though it doesn't capture technical implementation details.

Why This Matters to You

Problem it solves: Repetition and complexity when building large systems.

What you gain:

- Code reusability and structure
- Fewer bugs and cleaner logic
- Easier debugging and feature updates

Real-world context:

Every major framework (like Django, Flask, TensorFlow) relies on these principles to keep code organized and scalable.

2. The Foundation: Core Concepts Explained

Concept A: Inheritance

Definition:

Inheritance allows one class (child) to access and reuse attributes and methods from another class (parent).

Key characteristics:

- Promotes **code reuse**
- Forms a **hierarchical relationship**
- Supports **extension** of behavior

Example:

```
class Animal:
    def speak(self):
        print("Animal makes a sound")
```

```
class Dog(Animal):  
    def speak(self):  
        print("Dog barks")  
  
d = Dog()  
d.speak() # Output: Dog barks
```

Common confusion: Students often think inheritance copies code. It doesn't - it creates a **relationship** where the child **refers** to the parent's functionality.

 **Reference:** [Python Inheritance](#)

Concept B: Method Overriding

Definition: When a **child class** redefines a method already present in the **parent class**, it's called method overriding.

How it relates to inheritance: Overriding modifies inherited behavior to suit specific needs.

Example:

```
class Bird:  
    def sound(self):  
        print("Chirp")  
  
class Parrot(Bird):  
    def sound(self):  
        print("Squawk!") # Overriding parent's method  
  
Parrot().sound()
```

Output:

```
Squawk!
```

 **Tip:** Overriding helps customize a general behavior into something specific.

 **Reference:** [Method Overriding](#)

Concept C: Method Overloading

Definition: Method overloading means using **one method name** to perform **different tasks** depending on input parameters.

But here's the truth:

Python **does not support true method overloading** like Java or C++. The **last defined method** with the same name overwrites the previous one.

How Python handles it instead:

We use **default parameters** or **args, kwargs** to simulate overloading.

Example:

```
class Calculator:
    def add(self, a, b=0, c=0):
        return a + b + c

calc = Calculator()
print(calc.add(5))          # 5
print(calc.add(5, 10))     # 15
print(calc.add(5, 10, 15)) # 30
```

Why Python avoids real overloading: Because Python is **dynamically typed** - it determines argument count and types at runtime, so strict method signatures aren't required.

 **Reference:** [Method Overloading](#)

Concept D: super()

Definition: `super()` allows the child class to call methods of its parent class without explicitly naming it.

Why it's useful:

- Avoids code duplication
- Makes inheritance chains cleaner

Example:

```
class Parent:
    def __init__(self, name):
        self.name = name

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name) # Calls Parent's __init__
        self.age = age

c = Child("Riya", 21)
print(c.name, c.age)
```

Output:

Riya 21

 **Tip:** Always use **super()** instead of directly calling "**Parent.__init__()**" for better flexibility.

 **Reference:** [Python Docs: super\(\) Function](#)

Concept E: Encapsulation

Definition: Encapsulation means **restricting direct access** to internal object data. It's done using `_` (protected) and `__` (private) prefixes.

Key characteristics:

- Protects sensitive data
- Encourages safe data modification
- Hides implementation details

Example:

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance # Private variable

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance
```

Common confusion: Beginners think `__balance` is fully hidden - but it's name-mangled (`_BankAccount__balance`) not invisible.

 **Reference:** [Encapsulation in Python](#)

Concept F: Abstraction

Definition: Abstraction hides internal implementation and exposes only what's necessary.

Example:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius ** 2
```

```
c = Circle(5)
print(c.area())
```

Output:

78.5

 **Reference:** [Abstraction in Python](#)

3. Seeing It in Action: Worked Examples

Example 1: Single Inheritance

```
class Vehicle:
    def start(self):
        print("Vehicle started")

class Car(Vehicle):
    def start(self):
        print("Car engine started")

Car().start()
```

Example 2: Multiple Inheritance

```
class Engine:
    def engine_type(self):
        return "Petrol"

class Body:
    def body_type(self):
        return "Sedan"

class Car(Engine, Body):
    pass

car = Car()
print(car.engine_type(), car.body_type())
```

Example 3: Method Overloading (Simulated)

```
class Printer:
    def print_data(self, *args):
        for arg in args:
```

```
print(arg)
```

```
Printer().print_data("Hello", "World", 123)
```

Example 4: Using super()

```
class Animal:
    def __init__(self, name):
        self.name = name
        print(f"{name} is an Animal")

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed
        print(f"{name} is a {breed}")
```

Example 5: Encapsulation

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.__salary = salary

    def show_salary(self):
        print(f"{self.name}'s salary is confidential.")
```

Example 6: Abstraction

```
from abc import ABC, abstractmethod

class Payment(ABC):
    @abstractmethod
    def pay(self):
        pass

class CreditCard(Payment):
    def pay(self):
        print("Payment done using Credit Card")

CreditCard().pay()
```

Example 7: Implementation of All OOP Concepts Together

```
from abc import ABC, abstractmethod

# Abstraction
class Person(ABC):
    @abstractmethod
    def introduce(self):
        pass

# Inheritance + Encapsulation
class Student(Person):
    def __init__(self, name, roll):
        self.name = name
        self.__roll = roll # Encapsulation

    def introduce(self):
        print(f"Hi, I'm {self.name}.")

# Inheritance + super()
class GraduateStudent(Student):
    def __init__(self, name, roll, thesis):
        super().__init__(name, roll)
        self.thesis = thesis

    # Method Overriding
    def introduce(self):
        super().introduce()
        print(f"My thesis topic is '{self.thesis}'.")

# Overloading Simulation
class Utility:
    def show(self, *args):
        print(" ".join(str(a) for a in args))

grad = GraduateStudent("Aditi", 42, "AI Ethics")
grad.introduce()
Utility().show("Welcome", grad.name, "to Advanced OOPs")
```

Output:

```
Hi, I'm Aditi.
My thesis topic is 'AI Ethics'.
Welcome Aditi to Advanced OOPs
```

4. Common Pitfalls: What Can Go Wrong and How to Avoid It

❌ Beginner Mistake	💡 Why It's a Problem	✅ Correct Approach
Forgetting to use super() in the child class	Causes uninitialized attributes from parent class	Always call super().__init__() to inherit setup
Accessing private data directly (obj.__balance)	Raises AttributeError	Use public getter/setter methods

❌ Beginner Mistake	💡 Why It's a Problem	✅ Correct Approach
Assuming Python supports true overloading	Later method overwrites earlier one	Use default args or args, kwargs instead
Confusing overriding with overloading	Conceptual misunderstanding	Overriding = child redefines; Overloading = same name, diff inputs
Over-using inheritance	Creates tight coupling between classes	Use composition when inheritance isn't natural

5. Your Turn: Practice & Self-Assessment

Practice Task:

Create a small system for a **Library** that:

- Has a base class **LibraryItem**
- Has a subclass **Book** with extra attribute **author**
- Has another subclass **Magazine** with attribute **issue_no**
- Use **super()** to call parent initialization
- Demonstrate encapsulation for **_title**
- Implement abstraction for **get_info()** method

Hint: Use **@abstractmethod** for **get_info()** and override it in child classes.

Check Your Understanding

Why does Python not support true method overloading?

What's the difference between overriding and overloading?

Why is encapsulation crucial for data security?

How does abstraction make complex systems easier to manage?

Answers & Explanations

Because Python determines method arguments dynamically at runtime.

Overriding changes inherited behavior; Overloading adapts to input types/counts.

It prevents accidental or unauthorized modification of internal data.

It allows you to work with simplified interfaces without seeing the underlying logic.

6. Consolidation: Key Takeaways & Next Steps

The Essential Ideas

- **Inheritance:** Reuse logic from parent classes.
- **Overriding:** Modify or extend inherited behavior.
- **Overloading:** Same name, different inputs (simulated in Python).
- **Encapsulation:** Hide data; control access through methods.
- **Abstraction:** Simplify systems by exposing only necessary details.

Mental Model Check

You should now think of OOP as a **blueprint for structuring relationships** - where classes represent entities, inheritance defines relationships, and encapsulation + abstraction protect and simplify interactions.

What You Can Now Do

You can design object-oriented architectures for real-world systems like banking, school, or e-commerce with cleaner, reusable, and secure Python code.

Additional Resources:

- [Inheritance and Composition](#)
- [Python Docs: Class and Instance Variables](#)
- [Programiz: Python OOP Concepts](#)

Quick Reference Card

Concept	Definition	Example Use Case	Code Snippet
Inheritance	Child class derives properties from parent	Dog inherits from Animal	class Dog(Animal): pass
Overriding	Redefine parent method in child	Changing speak() in Dog	def speak(self): print("Woof!")
Overloading	Same method name, diff inputs	Flexible argument functions	def add(self, a, b=0): ...
super()	Call parent method from child	Access parent __init__()	super().__init__(name)
Encapsulation	Hide internal data	Protect _balance	self.__balance

Concept	Definition	Example Use Case	Code Snippet
Abstraction	Expose essential info only	Abstract class with @abstractmethod	class Shape(ABC): ...



Motivational Note: Every great Python developer masters these OOP foundations. You've not only learned how objects work - you've learned how *systems think*. Keep practicing, keep building, and these principles will become second nature. 🚀